

SVM From Scratch

Classification Multi-Classes sur MNIST

Implémentation de l'algorithme SMO avec noyau RBF

TP — Apprentissage Automatique

INF 372 - Département d'Informatique— université de Yaoundé I

May 3, 2026

Noms	Matricule
KAMDOM NGUETCHESSI MERVEILLE PRISCA	23V2060
YOKO AYMERICK JEFFREY	23U2983
KENNE YEMTSA LOWEL RAYANN	23U2477

Plan

- 1 Introduction et Objectifs
- 2 Théorie du SVM
- 3 Le Noyau RBF
- 4 L'algorithme SMO
- 5 Classification Multi-Classes (OvO)
- 6 Implémentation et Résultats
- 7 Conclusion

Objectif du projet

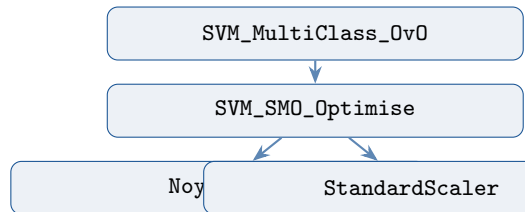
Objectif : implémenter un SVM complet *from scratch* pour classer les chiffres manuscrits MNIST (0–9).

Contrainte : aucune librairie ML

→ uniquement NumPy et Pandas

4 composants développés :

- `StandardScalerScratch` — normalisation
- `NoyauRBF` — similarité entre points
- `SVM_SMO_Optimise` — classifieur binaire
- `SVM_MultiClass_OvO` — extension 10 classes



Le dataset MNIST

Caractéristiques :

- 60 000 images d'entraînement
- 10 000 images de test
- Images 32×32 pixels (niveaux de gris)
- $\Rightarrow$ **1024 features** par image
- 10 classes : chiffres 0 à 9

Format CSV utilisé :

label	px_0	px_1	...	px_1023
7	0	0	...	255
2	0	84	...	0

Contrainte computationnelle :

La matrice de Gram $K \in \mathbb{R}^{n \times n}$ requiert :

$$n = 60\,000 \Rightarrow K \approx 27 \text{ Go !}$$

Sous-échantillonnage nécessaire

3 000 images entraînement

1 000 images test

Complexité du calcul de K : $\mathcal{O}(n^2 d)$

Idée fondamentale : trouver l'hyperplan à **marge maximale** séparant deux classes $y_i \in \{-1, +1\}$.

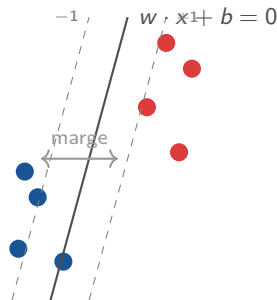
Problème (marge souple, C-SVM) :

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

$$\text{s.c. } y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Rôle du paramètre C

C grand	marge stricte, risque sur-apprentissage
C petit	marge large, plus robuste



Le SVM — Formulation duale et vecteurs supports

Par dualité de Lagrange, le problème devient :

$$\max_{\lambda} \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j K(x_i, x_j) \quad \text{s.c.} \quad 0 \leq \lambda_i \leq C, \quad \sum_i \lambda_i y_i = 0$$

La fonction de décision :

$$f(x) = \sum_{i \in \text{SV}} \lambda_i y_i K(x_i, x) + b$$

Conditions KKT (optimalité)

$\lambda_i = 0$	point bien classé, hors marge
$0 < \lambda_i < C$	point sur la marge (SV)
$\lambda_i = C$	point dans/hors la marge

Avantage clé

Seuls les **vecteurs supports** (SV, $\lambda_i > 0$)
contribuent à $f(x)$
 $\Rightarrow$ prédiction compacte et rapide.

L'astuce du noyau et le noyau RBF

Pourquoi un noyau ?

Les données réelles ne sont pas linéairement séparables. On les projette dans un espace de grande dimension via :

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

sans jamais calculer ϕ explicitement.

Noyau RBF (Gaussien) :

$$K_{\text{RBF}}(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

γ grand frontière locale, sur-apprentissage
 γ petit frontière lisse, généralisation

Calcul vectorisé de la matrice de Gram :

Identité utilisée :

$$\|x_i - x_j\|^2 = \|x_i\|^2 + \|x_j\|^2 - 2x_i^\top x_j$$

```
1 X1n = (X1**2).sum(1)[: , None]
2 X2n = (X2**2).sum(1)[None , :]
3 D    = X1n + X2n - 2*(X1 @ X2.T)
4 K     = np.exp(-g * np.maximum(D, 0))
```

Complexité : $\mathcal{O}(n^2d)$ sans boucle Python.

SMO — Sequential Minimal Optimization

Problème : la QP duale est de taille $n \times n$.

Un solveur générique : $\mathcal{O}(n^3)$ — inenvisageable.

Idée de Platt (1998) : décomposer en sous-problèmes de taille 2, résolubles analytiquement.

À chaque itération :

- 1 Choisir i : violation des conditions KKT
- 2 Choisir j : heuristique $\max |E_i - E_j|$
- 3 Optimiser analytiquement (λ_i, λ_j)
- 4 Mettre à jour b et le cache des erreurs

Critère d'arrêt : `max_passes` passes consécutives sans changement de λ .

Mise à jour analytique de λ_j

$$\eta = 2K_{ij} - K_{ii} - K_{jj} < 0$$

$$\lambda_j^{\text{new}} = \lambda_j^{\text{old}} - \frac{y_j(E_i - E_j)}{\eta}$$

$$\lambda_j^{\text{new}} \leftarrow \text{clip}(\lambda_j^{\text{new}}, L, H)$$

$$\lambda_i^{\text{new}} = \lambda_i^{\text{old}} + y_i y_j (\lambda_j^{\text{old}} - \lambda_j^{\text{new}})$$

Bornes L et H

$$\text{Si } y_i \neq y_j : L = \max(0, \lambda_j - \lambda_i)$$

$$\text{Si } y_i = y_j : L = \max(0, \lambda_i + \lambda_j - C)$$

$$H = \min(C, L + \lambda_i + \lambda_j)$$

Mise à jour du biais b (conditions KKT) :

$$b_1 = b - E_i - y_i \Delta \lambda_i K_{ii} - y_j \Delta \lambda_j K_{ij}$$

$$b_2 = b - E_j - y_i \Delta \lambda_i K_{ij} - y_j \Delta \lambda_j K_{jj}$$

$$0 < \lambda_i < C \quad b \leftarrow b_1$$

$$0 < \lambda_j < C \quad b \leftarrow b_2$$

$$\text{sinon} \quad b \leftarrow (b_1 + b_2)/2$$

Cache des erreurs $E_i = f(x_i) - y_i$:

Recalcul naïf à chaque étape : $\mathcal{O}(n^2)$ total.

Mise à jour **incrémentale** après chaque paire :

$$E \ += y_i \Delta \lambda_i K_{:,i} + y_j \Delta \lambda_j K_{:,j} + \Delta b$$

Gain de performance

Mise à jour en $\mathcal{O}(n)$ au lieu de $\mathcal{O}(n \cdot d)$ grâce à la matrice K pré-calculée.

Stratégie One-vs-One (OvO)

Comparaison des stratégies multi-classes :

Stratégie	Nb modèles	Déséquilibre
One-vs-All	K	fort
One-vs-One	$\binom{K}{2}$	faible

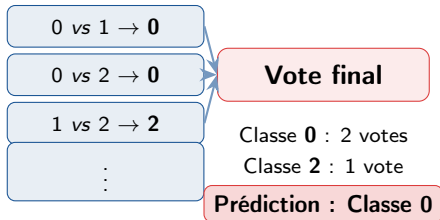
Pour MNIST ($K = 10$) :

$$\binom{10}{2} = 45 \text{ classifieurs binaires}$$

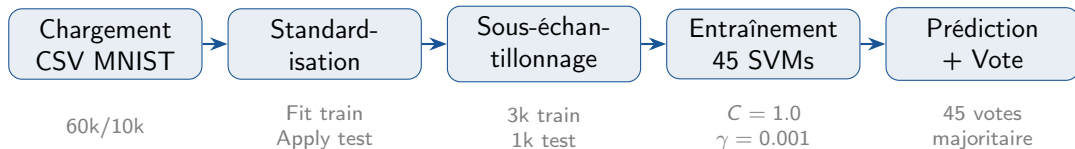
Chaque modèle est entraîné sur les données des **deux seules classes** concernées

⇒ sous-problèmes équilibrés et plus simples.

Phase de prédiction — vote majoritaire :



Pipeline complet



Hyperparamètres choisis :

Paramètre	Valeur
C	1.0
γ	0.001
max_passes	3
N_train	3 000
N_test	1 000

Complexités :

Étape	Complexité
Calcul de K	$\mathcal{O}(n^2 d)$
Stockage de K	$\mathcal{O}(n^2)$
Itération SMO	$\mathcal{O}(n)$
Prédiction	$\mathcal{O}(n_{SV} \cdot d)$

Résultat obtenu

$\approx 91.30\%$ de précision
(10 classes, 1 000 images test)

Comparaison :

Méthode	Précision
Notre SVM (5k)	$\approx 91.30\%$
SVM sklearn (60k)	$\approx 98\%$

Analyse des limites :

- SMO en Python pur : lent ($\sim 20-60$ min)
- Sous-échantillonnage réduit la précision
- Pas de validation croisée pour C , γ
- OvO non parallélisé (45 modèles séquentiels)

Ce qui fonctionne bien :

- Cache des erreurs : gain de vitesse réel
- Calcul vectorisé de K (NumPy)
- Heuristique KKT + $\max |E_i - E_j|$
- Prédiction compacte via vecteurs supports

Bilan :

Ce projet implémente de zéro l'ensemble de la chaîne SVM :

- Normalisation adaptative (features constantes)
- Noyau RBF vectorisé (identité matricielle)
- SMO avec cache incrémental des erreurs
- OvO avec vote majoritaire pour 10 classes

Résultat : **87%** de précision sur MNIST 10 classes, avec uniquement NumPy.

Code disponible avec notebook d'explication cellule par cellule.

Merci pour votre attention

Code	<code>svm_one_vs_one.py</code>
Notebook	<code>SVM_From_Scratch_MNIST.ipynb</code>
Docs	<code>README.md</code>

Questions ?